



PYTORCH

ว30263

การเขียนโปรแกรมปัญญาประดิษฐ์

PYTORCH

Library ที่ถูกพัฒนาขึ้นโดยมีต้นแบบจาก Library ชื่อว่า Torch มักถูกนำไปใช้งานในด้าน ML และ DL. PyTorch คือ open-source machine learning ที่ถูกพัฒนาขึ้นจากทีม Facebook's AI Research lab และเริ่มปล่อยให้ใช้งานในปี 2016

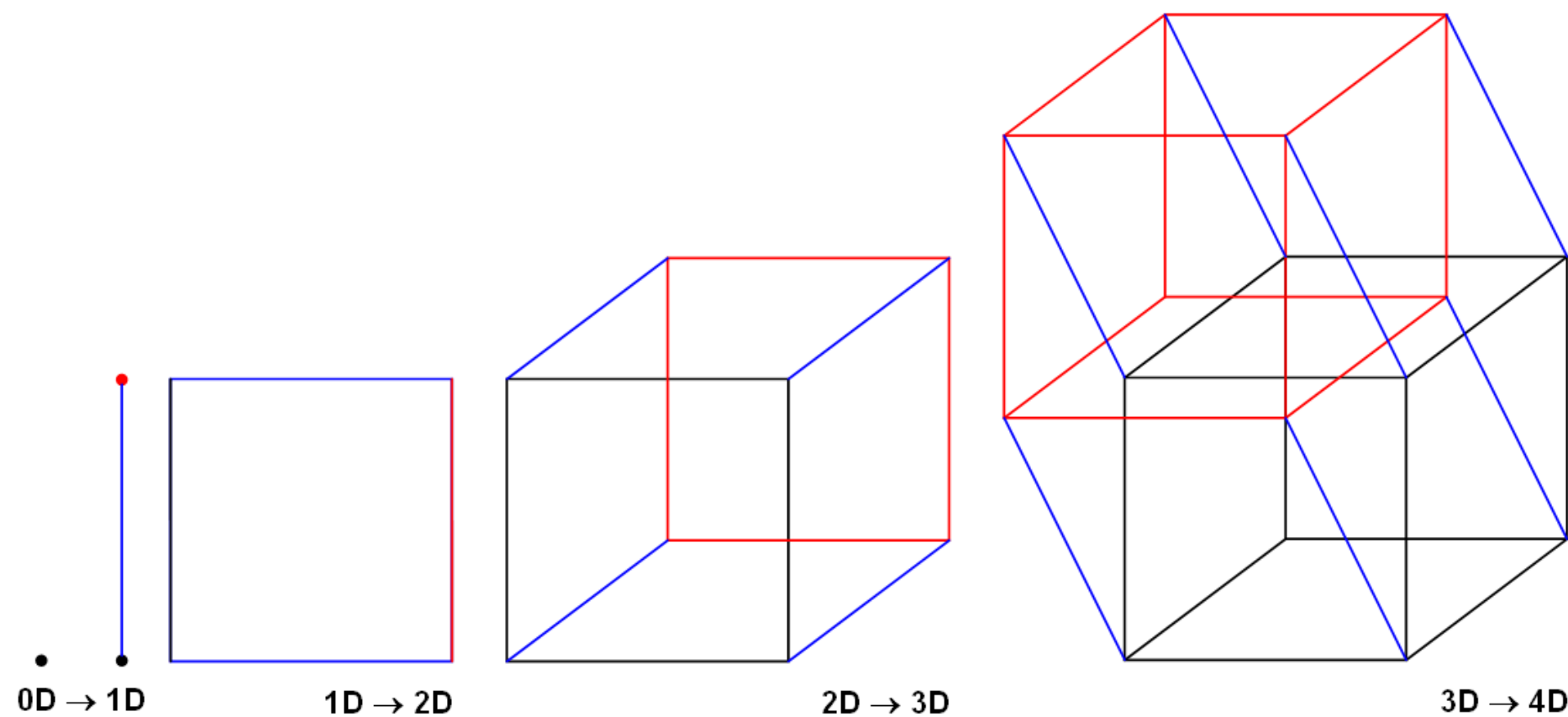


PY

TORCH

TENSOR

ในการที่จะคำนวณค่าใน PyTorch
ได้ เราจะทำการแปลงให้อยู่ในรูปของ
Tensor ซึ่ง **Tensor** ก็คือค่าที่เป็น
Vector กับ **Matrix** ในทาง
คณิตศาสตร์



TENSOR ของ PYTORCH



```
1 import torch
2 x = torch.rand(3,4,dtype= torch.float32)
3 print(x)
```

การสร้าง Tensor ที่มี Type เป็น
float32 ที่มีขนาดเป็น 3 Rows x 4
Columns แล้วทำการ Random ค่าใน
Tensor

TENSOR ของ PYTORCH



```
1 import torch
2 x = torch.zeros(5,3, dtype= torch.long)
3 print(x)
```

การสร้าง Tensor ที่มี Type เป็น long ที่มีขนาดเป็น 5 Rows x 3 Columns แล้ว
ทำการกำหนดค่า 0 ทุกตัวใน Tensor

TENSOR ของ PYTORCH



```
1 x = torch.zeros(5,3, dtype= torch.long)
2 x = x.new_ones(5,3,dtype=torch.float64)
3 print(x)
```

การสร้าง Tensor ที่มี Type เป็น
float64 ที่มีขนาดเป็น 5 Rows x 3
Columns แล้วทำการกำหนดค่า 0 ทุกตัว
ใน Tensor

TENSOR ของ PYTORCH



```
1 import torch
2 x = torch.tensor([4.123,3])
3 print(x)
```

การสร้าง Tensor ที่มี Type เป็น
float32 (หากเราไม่กำหนด Type โดย
Default แล้ว Type จะเป็น float32) ที่มี
ข้อมูลเป็น [4.123, 3]

TENSOR ของ PYTORCH



```
1 import torch
2 x = torch.zeros(5,3, dtype= torch.long)
3 x = torch.randn_like(x, dtype=torch.float32)
4 print(x)
```

การสร้าง Tensor ที่มี Type เป็น float32 ที่มีขนาดเหมือนกับ Tensor ที่ต้องการ (ในที่นี้คือตัวแปร x) แล้วทำการ Random ค่าใน Tensor

TENSOR ของ PYTORCH



```
1 import torch
2 x = torch.Tensor(5,3).fill_(7)
3 print(x)
```

การสร้าง Tensor ที่มี Type เป็น float32 (หากเราไม่กำหนด Type โดย Default แล้ว Type จะเป็น float32) ที่มี ขนาดเป็น 5 Rows x 3Columns แล้ว ทำการกำหนดค่า 7 ทุกตัวใน Tensor

TENSOR ของ PYTORCH



```
1 import torch
2 x = torch.Tensor(5,3).fill_(7).type(torch.int)
3 print(x)
```

หากเราต้องการเปลี่ยน Type Tensor เรา
สามารถใช้ `.type(torch.[type ที่
ต้องการเปลี่ยน])` ได้

TENSOR ของ PYTORCH



```
1 import torch
2 torch.manual_seed(1)
3 x = torch.randn(5,3)
4 print(x)
```

ในการ Random ค่า ใน Tensor แต่ละ
ครั้งไม่ต้องการให้ค่าเปลี่ยนแปลง เรา
สามารถกำหนดค่า **Seed** โดยการใช้คำสั่ง
`torch.manual_seed([ตัวเลขอะไร
ก็ได้])`

Press enter or click to view
image in full size

NUMPY

TENSOR

ในการใช้ Tensor บางครั้งต้องการเอาค่าใน
Numpy มาใส่ใน Tensor หรือเอาค่าใน Tensor
มาใส่ใน Numpy ซึ่งตัว Pytorch ก็มีเครื่องมือ

NUMPY กับ TENSOR



```
1 import torch
2 import numpy as np
3 a = np.array([1., 2., 3.])
4 t = torch.from_numpy(a)
5 print(a)
6 print(t)
7 t += 1
8 print(a)
9 print(t)
```

การแปลงค่าจาก numpy มาใส่ใน Tensor

จะใช้คำสั่ง `from_numpy` ([ตัวแปร numpy]) เพื่อทำการเอาค่าจาก numpy มาใส่ใน Tensor แต่วิธีนี้หากเรามีการแก้ไขค่าใน Tensor ค่าใน numpy จะมีการแก้ไขตามไปด้วย

NUMPY กับ TENSOR



```
1 import torch
2 import numpy as np
3 t = torch.tensor([1, 2, 3])
4 b = t.numpy()
5 print(b)
6 b += 1
7 print(t)
8 print(b)
```

การแปลงค่าจาก Tensor มาใส่ใน Numpy

เราจะใช้คำสั่ง `.numpy()` ในการแปลงค่าจาก Tensor มาเป็น numpy แต่หากเราใช้วิธีนี้จะทำให้ หากเรามีการแก้ไขค่าใน numpy ค่าใน Tensor ก็จะมีการแก้ไขตามไปด้วย

NUMPY กับ TENSOR



```
1 import torch
2 import numpy as np
3 t = torch.tensor([1, 2, 3])
4 b = t.numpy(t.data)
5 b += 1
6 print(t)
7 print(b)
```

การแปลงค่าจาก Tensor มาใส่ใน Numpy

เราจะใช้คำสั่ง `.numpy()` ในการแปลงค่าจาก Tensor มาเป็น numpy แต่หากเราใช้วิธีนี้จะทำให้ หากเรามีการแก้ไขค่าใน numpy ค่าใน Tensor ก็จะมีการแก้ไขตามไปด้วย

การคำนวณค่า ใน TENSOR

ถ้าต้องการ Dot Product Matrix เรา
สามารถทำได้โดยการใช้คำสั่ง
`torch.matmul([tensor1],
[tensor2])`



```
1  import torch
2  import numpy as np
3  t1 = torch.ones(5,3)
4  t2 = torch.Tensor(3,5).fill_(7)
5
6  print("t1",t1)
7  print("t2",t2)
8  print("dot product",torch.matmul(t1,t2))
```

การคำนวณค่า ใน TENSOR

นอกเหนือจากนั้น Pytorch ก็ยังให้เครื่องมือที่ช่วยคำนวณ ค่าเฉลี่ย, ผลรวม, ค่าเบี่ยงเบน, หาค่า max, หาค่า min และอื่นๆอีกเพียบ



```
1 import torch
2 import numpy as np
3 t = torch.ones(5,3)
4 print(t.mean())
5 print(t.sum())
6 print(t.std())
7 print(t.min())
8 print(t.max())
```

การคำนวณค่า ใน TENSOR

บางครั้งเราต้องการเปลี่ยนขนาดของ Tensor จากที่มีอยู่ให้เป็นขนาดที่ต้องการเช่น ต้องการเปลี่ยน Tensor ขนาด 5×3 ให้เป็นขนาด 1×15 หรือ 3×5 ซึ่งมีข้อแม้ว่าในการเปลี่ยนขนาดควรมีจำนวนข้อมูลทั้งหมดเท่ากัน เช่น จาก 5×3 คือ มีทั้งหมด 15 ค่า เราสามารถเปลี่ยนเป็นขนาดอะไรก็ได้ที่มีจำนวนข้อมูลทั้งหมด 15 ค่าเหมือนกัน เช่น 1×15 และ 3×5



```
1  import torch
2  import numpy as np
3  t = torch.rand(5,3)
4  print(t)
5  print(t.view(1,15))
6  print(t.view(3,5))
7  print(t.flatten())
8
```

การใช้ GPU

การทำให้ Pytorch สามารถคำนวณ Tensor โดยใช้ GPU ได้นั้นก็จะมีการเขียนเพิ่มเติมนิดนึง เราจะต้องประกาศ device ก่อนเสร็จแล้วเลือก device เป็น “cuda” หลังจากนั้นเวลาใช้ Tensor ก็ทำการใส่คำสั่ง .to(device) ได้เลย หลังจากนั้นเมื่อมีการคำนวณค่า Tensor ที่เติมด้วย .to(device) ก็จะทำการคำนวณใน GPU กันที่



```
1 import torch
2 import numpy as np
3 device = torch.device("cuda" if torch.cuda.is_available()
4                       else "mps" if torch.mps.is_available()
5                       else "cpu")
6 print("Using device:", device)
7 t = torch.rand(5,3).to(device)
8 print(t)
```

การหาอนุพันธ์

ใน PyTorch นั้นได้ทำส่วนของการคำนวณอนุพันธ์ ให้เรา โดยการที่จะทำให้ Pytorch คำนวณอนุพันธ์ให้เราได้นั้น ก่อนอื่นจำเป็นต้องตั้งค่า `.requires_grad` ให้เป็น `True` เพราะหากไม่กำหนดค่า โดย Default จะเป็น `False` และจะไม่สามารถคำนวณอนุพันธ์ได้

เมื่อตั้งค่า `.requires_grad = True` เรียบร้อย เราสามารถคำนวณอนุพันธ์ได้โดยการพิมพ์คำสั่ง `.backward` เพื่อคำนวณ แล้วค่าอนุพันธ์ที่คำนวณได้ จะอยู่ในแอตทริบิวต์ที่ชื่อว่า `.grad`

การหาอนุพันธ์

สร้างตัวแปร x และ ตัวแปร w เพื่อมาทำการ dot product กัน แล้วเราก็ใส่คำสั่ง `y.backward()` เพื่อต้องการหาอนุพันธ์ y เทียบกับ w (เนื่องจากเราได้ใช้คำสั่ง `w.requires_grad = True` ไป) ดังนั้นอนุพันธ์ y เทียบกับ w จึงต้องมีค่าเท่ากับ x มาเก็บไว้ในแอตทริบิวต์ `w.grad`

```
1 import torch
2 import numpy as np
3 device = torch.device("cuda" if torch.cuda.is_available()
4                       else "mps" if torch.mps.is_available()
5                       else "cpu")
6 print("Using device:", device)
7 x = torch.ones(1, 5).to(device)
8 w = torch.Tensor(5, 1).fill_(2.0).to(device)
9
10 w.requires_grad = True
11 y = torch.matmul(x, w)
12
13 y.backward()
14 print(y.grad)
15 print(w.grad)
16 print(x.grad)
```

การหาอนุพันธ์



```
1 import torch
2 import numpy as np
3 device = torch.device("cuda" if torch.cuda.is_available()
4                       else "mps" if torch.mps.is_available()
5                       else "cpu")
6 print("Using device:", device)
7 x = torch.ones(1, 5).to(device)
8 w = torch.Tensor(5, 1).fill_(2.0).to(device)
9
10 w.requires_grad = True
11 y = torch.matmul(x, w)
12
13 y.backward()
14 print(y.grad)
15 print(w.grad)
16 print(x.grad)
```

กำหนดให้

$$x = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

$\rightarrow X = \text{torch.ones}(1,5)$

$$w = [2 \ 2 \ 2 \ 2 \ 2] \rightarrow W = \text{torch.Tensor}(5,1).\text{fill_}(2)$$

$$y = w \cdot x \rightarrow y = \text{torch.matmul}(x,w)$$

$y.backward()$

$$\frac{\partial y}{\partial w} = x = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

$w.requires_grad = \text{True}$

■ สมการคณิตศาสตร์

■ การเขียนโค้ด